

Instructions: First, ensure that all files except the PDFs are in the folder you are using for testing. Then, you can start the server.js with the usual 'npm start' instruction. Once the website has been opened in the browser click create tables to make all the tables, though they will all be empty for now. Note that the observe buttons are there to show each table individually. Then click Init lookups and it will make ten rows for the tables Category, Vehicle, UserInfo, Driver, and BankAccount. After it is strongly encouraged to press the Front Desk Demo button, as you will see the transaction actively add rows to payment, ride, and companyCommission, also allows you to see clearly that it updates a row in bankAccount. Press the stimulation(100) button to simulate 100 transactions being processed. The report 1, 2, and 3 buttons are all different queries made using join; you can see below exactly what they do; however, it should be evident if it works. The clear log button will get rid of the rows. The database itself gets saved in a folder called database. Otherwise, nothing more to say.

Notes: The user info table was turned into app_user in the code

Create Tables - Makes 8 Tables: CompanyCommission, Ride, Payment, BankAccount, Driver, Vehicle, Category, UserInfo with all attributes associated with them; however, it should still be empty.

Init Lookups - Will fill the tables Category, Vehicle, UserInfo, Driver, and BankAccount with ten rows using faker for certain char attributes like the model attribute in the Vehicle table

Front Desk Demo - Is the transaction, it should make a selected amount depending on the base_fare and per_km times each other, along with tax calculations. Once it has calculated that value and is stored, it inserts a row to the payment table based on the UserInfo and BankAccount tables (Basically takes the user_id and bank_acct_id to make the new payment). Then it updates the BankAccount row with the specific bank_acct_id used for payment, insertion, and will update the payment attribute to either subtract from the balance attribute or add to the balance attribute, depending on the method (Credit: adds, Debit: subtracts). Afterward, it inserts a new row into the ride table, then inserts a new row into the CompanyCommission table to show this new ride taking place, sharing any previous values before.

Stimulation - Simply performs the front desk demo a hundred times

Clear Log - Gets rid of all the rows

Observe _____ - All observes are there to allow to see each individual table

Report 1 - (Query 1) Groups by the user ID, name, and month to show how many rides and the total amount they all cost together there have been with those particular attributes

Report 2 - (Query 2) Groups by the driver ID, driver name, and month to show the gross (total earnings), commission (The company takes for the branding), and total net, which is simply the gross minus the commission.

Report 3 - (Query 3) Groups by the category type and start location to show the total rides, revenue (total earned), and tax collected (total of all those rides' tax amounts)